

SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Concept Mapping

COURSE NAME: Data Structures

COURSE CODE: CSA0309

Register Number	192524434
Name	Gurucharan M

COURSE OUTCOME COVERED

CO1: Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2, BL3)

Assessment Tool Weightage: 35%

Questions:2

EACH QUESTIONS CARRIES: 10 MARKS

Total Marks:20

Duration: ONE HOUR

Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

Code of Conduct:

I Gurucharan M/192524434 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Gurucharan M

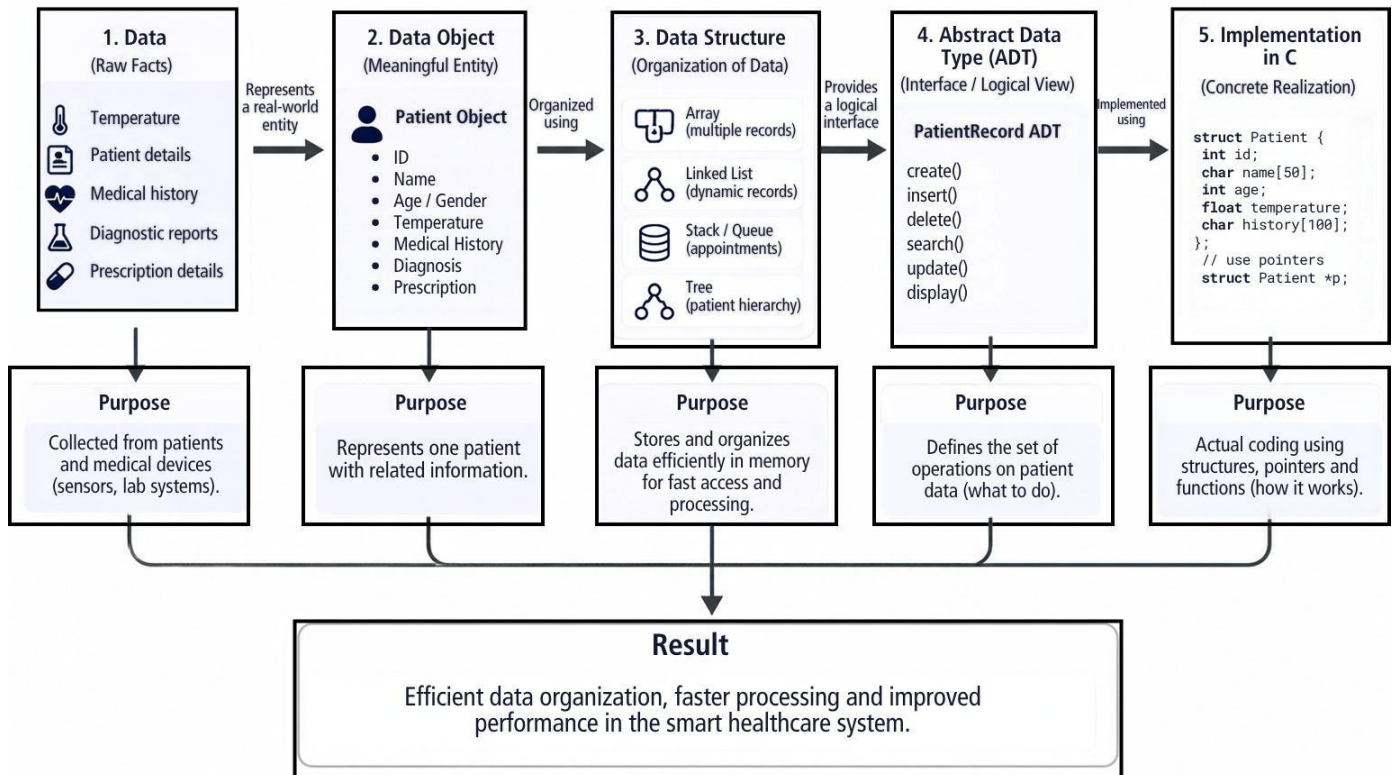
RUBRICS FOR CALCULATION:

Concept Mapping					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

1.

Smart Healthcare System – Concept Map

(Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C)



Smart City Traffic Management System – Concept Map

Q1 — Explanation and Justification

Data: Raw facts such as a temperature value, patient identifier, blood-pressure reading, or diagnostic value. Data becomes useful when it is given context and organized.

Data Object: A meaningful collection or instance of related data. For example, a patient record can combine identification, vital signs, medical history, and diagnostic information.

Data Structure: A systematic way to organize and store data so that operations such as access, insertion, deletion, searching, and updating can be performed efficiently. Arrays and linked lists are relevant choices for patient-record collections.

Abstract Data Type (ADT): Defines what data and operations are supported without specifying how they are implemented. An ADT separates the logical behavior of a data collection from its implementation.

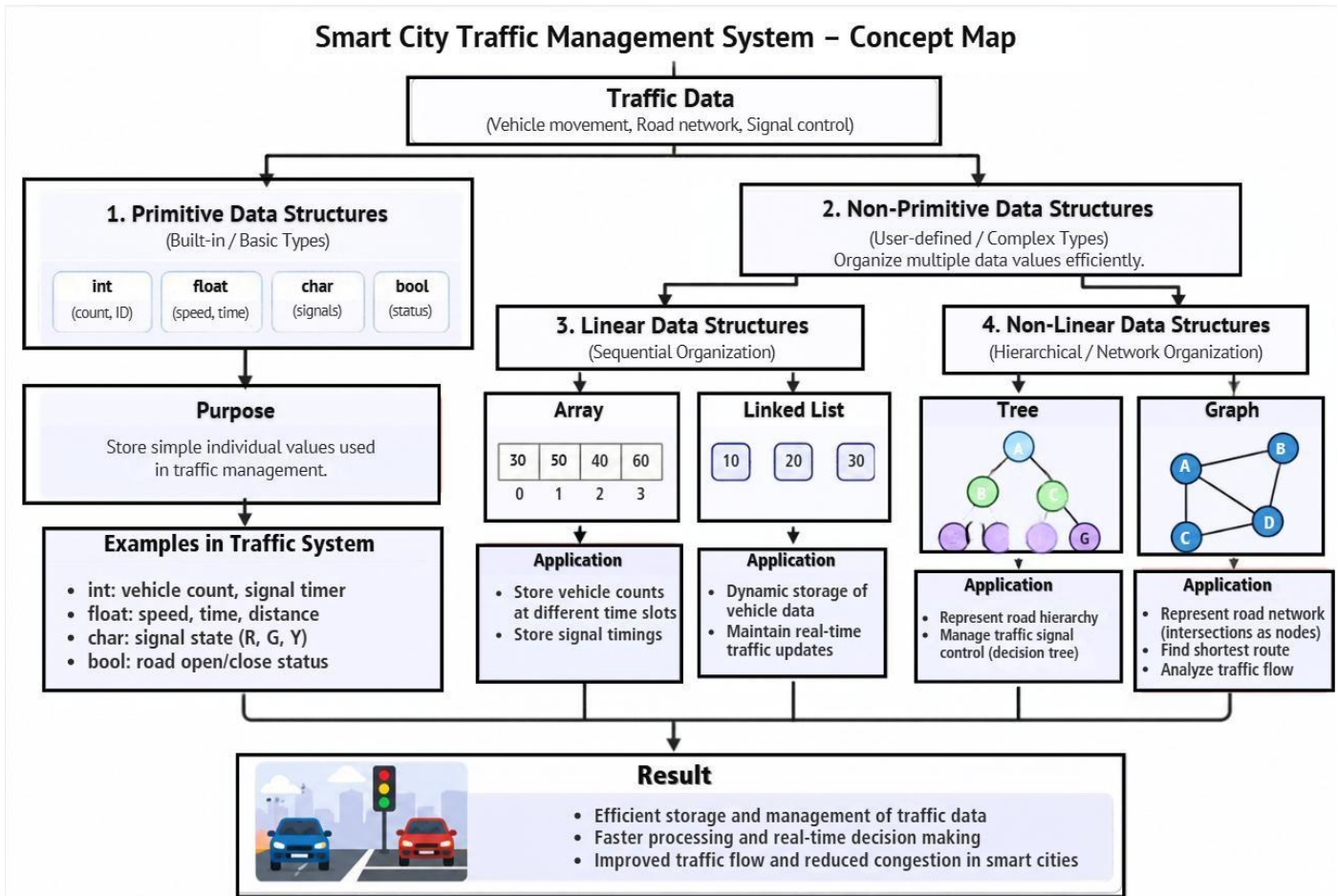
Implementation in C: C provides structures, pointers, arrays, dynamic memory, and functions that can implement the behavior defined by an ADT.

Structures: A C struct can group heterogeneous patient attributes into a single record, improving organization and readability.

Pointers: Pointers allow indirect access to memory and support dynamic data structures such as linked lists. They can reduce unnecessary copying when records or nodes are manipulated.

Operations: Typical operations include create, insert, delete, search, access, and update. The selected data structure determines how efficiently these operations can be performed.

2.



Q2 — Classification, Relationships and Applications

Primitive Data Structures: Basic built-in data types such as int, char, float, and double. In a traffic system they can represent quantities such as vehicle counts, signal timings, identifiers, and measured speeds.

Non-Primitive Data Structures: Structures built to organize collections or relationships among data. They include linear and non-linear structures.

Linear Structures: Elements are arranged sequentially. Arrays and linked lists are common examples.

Array: Stores elements in contiguous indexed positions. It is suitable for fixed or predictable collections such as vehicle records for a defined road segment, and provides direct indexed access.

Linked List: Stores elements as dynamically connected nodes. It is useful when the number of active vehicles changes frequently, because nodes can be inserted or removed without requiring a contiguous block of memory.

Non-Linear Structures: Elements can have hierarchical or network relationships rather than one simple sequence.

Tree: Represents hierarchical relationships. A traffic system can use tree-like organization for road hierarchies, zones, intersections grouped by regions, or hierarchical classification of routes.

Graph: Represents entities as vertices and relationships as edges. A road network is naturally modeled as a graph: intersections can be vertices and roads can be edges. Graph traversal and shortest-path techniques can support route analysis.